[TÊN TRƯỜNG / HỌC VIỆN]

[TÊN KHOA / VIỆN]

BÁO CÁO BÀI THỰC HÀNH 6

GIAO TIẾP PHI NGÔN NGỮ VÀ ĐIỀU KHIỂN TURTLEBOT 4 BẰNG CỬ CHỈ TAY

Non-verbal Human–Robot Interaction
Using Hand Gesture Recognition

Học phần:	[Tên học phần]
Giảng viên hướng dẫn:	[Họ và tên giảng viên]
Sinh viên thực hiện:	[Họ và tên sinh viên]
Mã số sinh viên:	[Mã số sinh viên]
Lớp / Nhóm:	[Thông tin lớp / nhóm]

[Địa điểm], tháng [. . .] năm 2026

Mục lục

1 Giới thiệu bài toán **1**
1.1 Đặt vấn đề . . . 1
1.2 Mục tiêu và yêu cầu thực hiện . . . 1
1.3 Phạm vi và sản phẩm thực hiện . . . 1

2 Kiến trúc và thiết kế hệ thống **1**
2.1 Sơ đồ kiến trúc tổng thể . . . 1
2.2 Chức năng các node . . . 2
2.3 Topic và kiểu dữ liệu . . . 2
2.4 Luồng xử lý một yêu cầu điều khiển . . . 3

3 Nhận diện cử chỉ tay bằng MediaPipe **3**
3.1 Thu nhận và tiền xử lý ảnh . . . 3
3.2 Cấu hình mô hình và cử chỉ tĩnh . . . 3
3.3 Nhận diện Wave theo thời gian . . . 4
3.4 Lọc nhiễu, ảnh cũ và xuất kết quả . . . 4

4 Thiết kế điều khiển hành vi **5**
4.1 Ánh xạ cử chỉ và hành vi robot . . . 5
4.2 Máy trạng thái và cơ chế xác nhận . . . 5
4.3 Thực hiện chuyển động với phản hồi odometry . . . 6
4.4 Watchdog và điều kiện sẵn sàng . . . 6
4.5 Kiểm tra vật cản và giới hạn của cơ chế dừng . . . 6

5 Triển khai mã nguồn và vận hành **7**
5.1 Môi trường thực hiện . . . 7
5.2 Cấu trúc mã nguồn và chức năng từng file . . . 7
5.3 Build và khởi chạy từng terminal . . . 7
5.4 Kiểm tra các topic và thông số . . . 8

6 Kết quả thực nghiệm và đánh giá **9**
6.1 Kịch bản, cách thu dữ liệu và giới hạn đánh giá . . . 9
6.2 Kết quả nhận diện sáu cử chỉ . . . 9
6.3 Chọn hành vi và xác nhận trước khi chuyển động . . . 11
6.4 Bốn chu kỳ tiến–lùi . . . 11
6.5 Quỹ đạo vòng tròn . . . 12
6.6 Động tác chào từ Wave . . . 13
6.7 Dừng, hủy và mất tín hiệu . . . 13
6.8 Bảng tổng hợp kết quả và các lần gián đoạn . . . 13
6.9 Phân tích vấn đề đã gặp và cách xử lý . . . 14
6.10 Đánh giá kiểm thử và phạm vi kết luận . . . 15

7 Kết luận và hướng phát triển **15**
7.1 Kết quả đạt được . . . 15
7.2 Hạn chế và hướng cải tiến . . . 15

Tài liệu tham khảo **16**

Phụ lục: Bộ minh chứng thực nghiệm **16**

Chương 1. Giới thiệu bài toán

1.1. Đặt vấn đề

Cử chỉ tay cho phép người dùng truyền đạt ý định điều khiển bằng hình ảnh thay vì nhập lệnh trực tiếp. Trong bài thực hành này, webcam laptop quan sát bàn tay, phần mềm nhận diện xác định cử chỉ và ROS 2 chuyển kết quả đến bộ điều khiển TurtleBot 4 trong Gazebo. Bài toán không chỉ là gán một nhãn cho ảnh: hệ thống còn phải phân biệt thao tác chọn và thao tác xác nhận, ngăn lệnh bị lặp do giữ tay, đồng thời dừng chuyển động khi tín hiệu điều khiển không còn hợp lệ.

Một bàn tay được phát hiện chưa đồng nghĩa với một lệnh đã được chấp nhận. Các điểm đặc trưng có thể vẫn xuất hiện trong khi bộ phân loại trả về `None`. Vì vậy, thiết kế tách riêng nhận diện hình ảnh và quyết định hành vi; chỉ kết quả đạt điều kiện mới được sử dụng để duy trì hoặc kích hoạt chuyển động.

1.2. Mục tiêu và yêu cầu thực hiện

Hệ thống cần nhận diện tối thiểu năm cử chỉ; bản triển khai sử dụng sáu cử chỉ gồm Wave, Pointing Up, Victory, Thumb Up, Open Palm và Closed Fist. Hai node Python phải giao tiếp qua `/detected_gesture`, điều khiển qua `/cmd_vel`, hiển thị kết quả trên webcam và thực hiện các chuyển động trong sân mô phỏng đủ rộng.

Các yêu cầu chức năng chính là: chọn hành vi nhưng chưa chạy ngay; chỉ thực hiện sau xác nhận; hoàn thành bốn chu kỳ tiến–lùi, một vòng tròn và động tác chào; dừng bằng bàn tay mở; hủy bằng nắm tay; dừng khi mất tín hiệu. Trong phần demo, người dùng không dùng bàn phím hoặc công cụ teleop để lái robot.

1.3. Phạm vi và sản phẩm thực hiện

Sản phẩm gồm package `turtlebot4_gesture_control` [8], cấu hình sân `gesture_arena`, môi trường MediaPipe riêng, video demo ngày 24/10/2026, ảnh thực nghiệm và dữ liệu odometry. Robot cùng các thành phần mô phỏng nền tảng được tái sử dụng từ các package TurtleBot 4 chính thức [6]; bài này không xây dựng lại controller bánh xe và không sử dụng Nav2 để thực hiện ba hành vi.

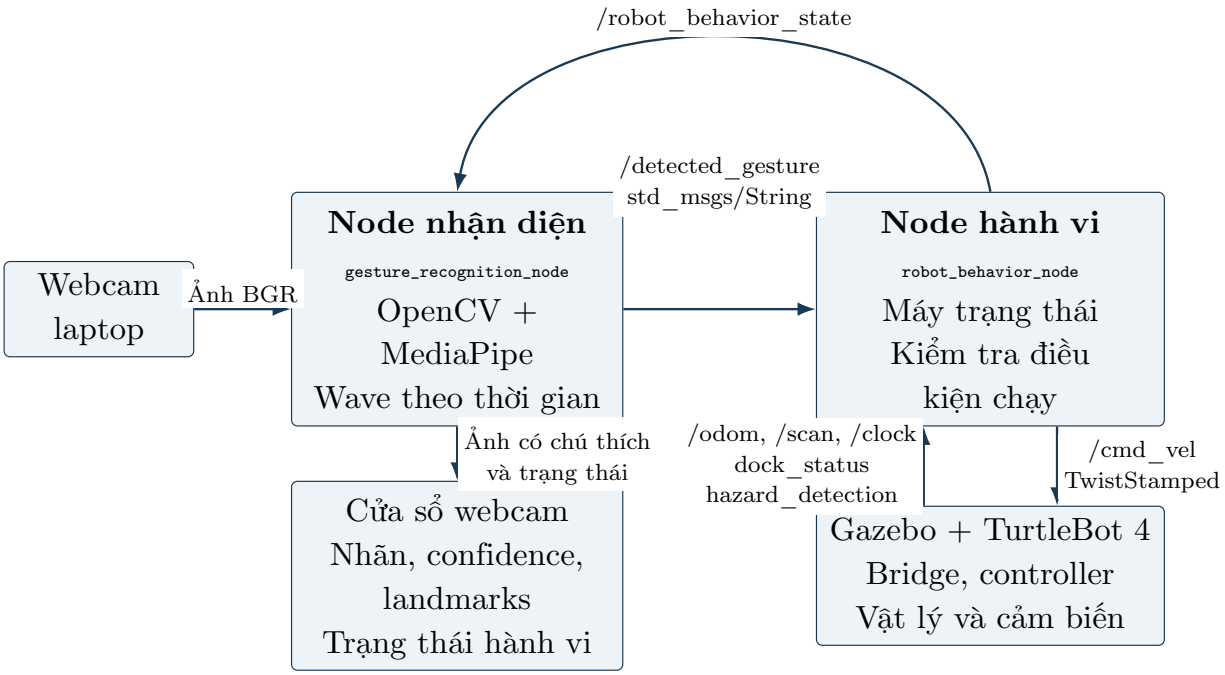
Báo cáo đánh giá một lượt demo với một người dùng trong một điều kiện webcam. Các kết quả đủ để minh chứng chức năng của lượt chạy, nhưng chưa phải bộ dữ liệu đánh giá độ chính xác trên nhiều người, nhiều góc nhìn hoặc nhiều mức chiếu sáng.

Chương 2. Kiến trúc và thiết kế hệ thống

2.1. Sơ đồ kiến trúc tổng thể

Hình 1 thể hiện hai node ứng dụng và chiều trao đổi dữ liệu. OpenCV, MediaPipe và thuật toán Wave nằm bên trong node nhận diện, không phải các ROS node độc lập. Khối mô phỏng gộp robot, bridge và các thành phần controller có sẵn.

ROS 2 cho phép một node vừa publish vừa subscribe nhiều topic [4]. Trong thiết kế này, chiều nhận diện–điều khiển truyền một chuỗi ngắn, còn ảnh có chú thích được publish riêng để quan sát. Nhờ đó, việc hiển thị ảnh không thay đổi định dạng lệnh điều khiển.



Hình 1: Kiến trúc hệ thống và các luồng điều khiển, phản hồi chính.

2.2. Chức năng các node

Node	Xử lý chính	Đầu ra
`gesture_recognition_node`	Đọc webcam; nhận diện một bàn tay; lọc confidence; phát hiện Wave; hiển thị ảnh.	Cử chỉ, ảnh có chú thích và thông tin nhận diện.
`robot_behavior_node`	Lọc cử chỉ theo thời gian; quản lý lựa chọn và xác nhận; kiểm tra cảm biến; đo tiến độ.	Vận tốc có timestamp và trạng thái hành vi.
`lab06_demo_observer`	Chỉ theo dõi topic trong lượt quay; lưu sự kiện và odometry, nhắc các bước demo.	Log, ảnh và dữ liệu đánh giá; không gửi lệnh robot.

Bảng 1: Hai node vận hành chính và node quan sát phục vụ thu thập minh chứng.

Node quan sát là công cụ ghi thực nghiệm, không nằm trong chuỗi quyết định vận tốc. Mọi cử chỉ trong video đều đến từ webcam của người dùng; không có thao tác publish cử chỉ giả để thay thế nhận diện thật.

2.3. Topic và kiểu dữ liệu

Kiểu dữ liệu trên `/cmd_vel` của môi trường đang dùng là `TwistStamped`, không suy diễn từ ví dụ sử dụng `Twist` ở môi trường khác. Message gồm header và trường twist [5]; node gán timestamp theo đồng hồ mô phỏng, frame `base_link`, sử dụng `linear.x` và `angular.z`. Trạng thái hành vi có QoS transient local để cửa sổ webcam mới kết nối vẫn nhận được trạng thái gần nhất; odom, scan và hazard dùng cấu hình QoS sensor data.

Topic	Kiểu message	Chức năng
/detected_gesture	std_msgs/msg/String	Nhãn cử chỉ hoặc None.
/cmd_vel	geometry_msgs/msg/TwistStamped	Vận tốc thẳng và góc kèm timestamp.
/odom	nav_msgs/msg/Odometry	Phản hồi vị trí và hướng.
/scan	sensor_msgs/msg/LaserScan	Khoảng cách theo hướng quét.
/clock	rosgraph_msgs/msg/Clock	Thời gian mô phỏng.
/dock_status	irobot_create_msgs/msg/DockStatus	Trạng thái đang ở dock.
/hazard_detection	irobot_create_msgs/msg/HazardDetectionVector	Theo dõi bumper.
/robot_behavior_state	std_msgs/msg/String	JSON trạng thái, tiến độ, ready.
/gesture_recognition/image	sensor_msgs/msg/Image	Ảnh BGR đã chú thích.
/gesture_recognition/debug	std_msgs/msg/String	Nhãn gốc, confidence, thời gian xử lý.

Bảng 2: Các topic được sử dụng trong bản triển khai ROS 2 Jazzy.

2.4. Luồng xử lý một yêu cầu điều khiển

Một thao tác đi vòng tròn diễn ra theo trình tự: người dùng giơ Victory; node nhận diện publish `Victory`; node hành vi chờ cử chỉ ổn định và chuyển sang `SELECTED`; người dùng giơ Thumb Up; khi xác nhận ổn định và `ready=True`, node tạo hành vi vòng tròn, chuyển sang `RUNNING` và gửi vận tốc. Odom được tích lũy đến khi đủ một vòng, sau đó lệnh vận tốc về không và trạng thái thành `COMPLETED`.

Trong suốt quá trình, các điều kiện dừng vẫn được kiểm tra. Luồng phản hồi trạng thái về webcam giúp người dùng biết sự khác biệt giữa "đã chọn", "đã xác nhận" và "đã hoàn thành", thay vì suy đoán chỉ từ vị trí robot trên màn hình.

Chương 3. Nhận diện cử chỉ tay bằng MediaPipe

3.1. Thu nhận và tiền xử lý ảnh

Webcam được mở bằng `cv2.VideoCapture` với chỉ số camera 0 và backend V4L2. OpenCV cung cấp cơ chế kiểm tra mở camera và đọc từng khung hình [3]; bản triển khai yêu cầu ảnh 640×480, giới hạn bộ đệm một ảnh và lật gương để thao tác trực quan. Ảnh BGR được chuyển sang RGB trước khi tạo đối tượng `mp.Image`.

Vòng capture và suy luận chạy trong một luồng riêng. Timer ROS chỉ đọc kết quả mới nhất, không tạo hàng đợi dài các ảnh cũ. Cách tổ chức này giúp việc publish heartbeat và cập nhật giao diện không phải chờ trực tiếp một lời gọi suy luận đang xử lý. Nếu lỗi camera, node kết thúc; bộ điều khiển nhận ra mất tín hiệu qua watchdog.

3.2. Cấu hình mô hình và cử chỉ tĩnh

MediaPipe Gesture Recognizer trả về nhóm nhãn, landmarks và handedness [1]. Chương trình dùng model `gesture_recognizer.task`, chế độ `VIDEO`, một bàn tay và timestamp mili

giây tăng nghiêm ngặt. Lời gọi `recognize_for_video` xử lý đồng bộ trong luồng capture; đây không phải chế độ callback `LIVE_STREAM` [2].

Hệ thống lấy nhãn đứng đầu và chỉ sử dụng khi có landmarks, confidence đạt 0,65 và nhãn thuộc nhóm cho phép. Năm nhãn tĩnh phục vụ bài là `Pointing_Up`, `Victory`, `Thumb_Up`, `Open_Palm` và `Closed_Fist`. Các nhãn không phục vụ điều khiển, kết quả không chắc chắn hoặc không có tay đều tạo đầu ra `None`.

Listing 1: Đoạn xử lý đầu vào và suy luận được sử dụng trong node.

```
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
timestamp = max(last_timestamp + 1, int(captured * 1000))
result = recognizer.recognize_for_video(image, timestamp)
```

Có 21 landmarks; chương trình vẽ các liên kết giữa những điểm này để người dùng quan sát bàn tay. Tuy nhiên, landmarks còn xuất hiện không có nghĩa cử chỉ đã đạt điều kiện điều khiển: nhãn có thể là `None` hoặc confidence chưa đạt ngưỡng.

3.3. Nhận diện Wave theo thời gian

Wave không nằm trong các nhãn dựng sẵn của model đang dùng [1]. Hệ thống xây dựng cử chỉ động này bằng cách kết hợp nhãn AI `Open_Palm` với chuyển động ngang qua nhiều khung hình. Không có bước huấn luyện một mô hình Wave riêng trong bài thực hành.

Đặc trưng vị trí được lấy từ trung bình tọa độ ngang chuẩn hóa của bốn đầu ngón tay:

$$x_h(t) = \frac{x_8(t) + x_{12}(t) + x_{16}(t) + x_{20}(t)}{4}. \tag{1}$$

Việc sử dụng đầu ngón tay thay cho chỉ vùng gần cổ tay cho phép ghi nhận cả kiểu vẫy quanh một cổ tay gần như cố định. Trên cửa sổ lịch sử tối đa 3 giây, điều kiện biên độ là

$$A = \max x_h(t) - \min x_h(t) \geq 0{,}09. \tag{2}$$

Đây là tỷ lệ theo chiều rộng ảnh, không phải đơn vị mét hoặc centimét ngoài thực tế. Cần thêm ít nhất hai lần đảo chiều, có hysteresis 0,025 để không đếm các dao động nhỏ thành hướng mới. Một lần quét ngang hoặc bàn tay mở đứng yên không đủ điều kiện thành Wave.

Lịch sử bị làm mới khi đổi handedness, khoảng cách giữa hai mẫu quá lớn hoặc vị trí nhảy đột ngột. Một quan sát không chắc chắn ngắn được giữ trong lịch sử, nhưng khung hình đó vẫn publish `None`. Nếu không có dịch chuyển đáng kể trong hơn 0,25 giây, lịch sử Wave bị xóa; bàn tay mở đứng yên trở lại `Open_Palm`. Điểm hiển thị khi nhận Wave là confidence của Open Palm kết hợp điều kiện chuyển động, không phải xác suất của một bộ phân loại Wave được huấn luyện độc lập.

3.4. Lọc nhiễu, ảnh cũ và xuất kết quả

Timer nhận diện publish ở chu kỳ 0,1 giây. Nếu ảnh gần nhất đã cũ hơn 0,35 giây, đầu ra bị đổi thành `None` thay vì tiếp tục sử dụng một lệnh cũ. Cơ chế này khác với chỉ kiểm tra node còn sống: một node vẫn chạy nhưng không nhận ảnh mới không được duy trì chuyển động vô thời hạn.

Ngưỡng confidence cao có thể giảm nhận nhầm nhưng tăng bỏ sót; ngưỡng thấp giúp tiếp nhận nhiều khung hình hơn nhưng dễ chấp nhận hình tay không rõ. Vì vậy, ngưỡng 0,65

được phối hợp với điều kiện giữ ổn định ở node hành vi, thay vì dùng một frame đơn lẻ để kích hoạt chuyển động.

Giao diện hiển thị nhãn, confidence, landmarks và trạng thái robot. Ảnh đã chú thích được publish dưới dạng `sensor_msgs/Image`. Tần số publish 10 Hz, giới hạn vòng capture 15 fps và tốc độ video ghi màn hình 20 fps là ba đại lượng khác nhau; báo cáo không dùng chúng để suy ra FPS suy luận thực tế.

Chương 4. Thiết kế điều khiển hành vi

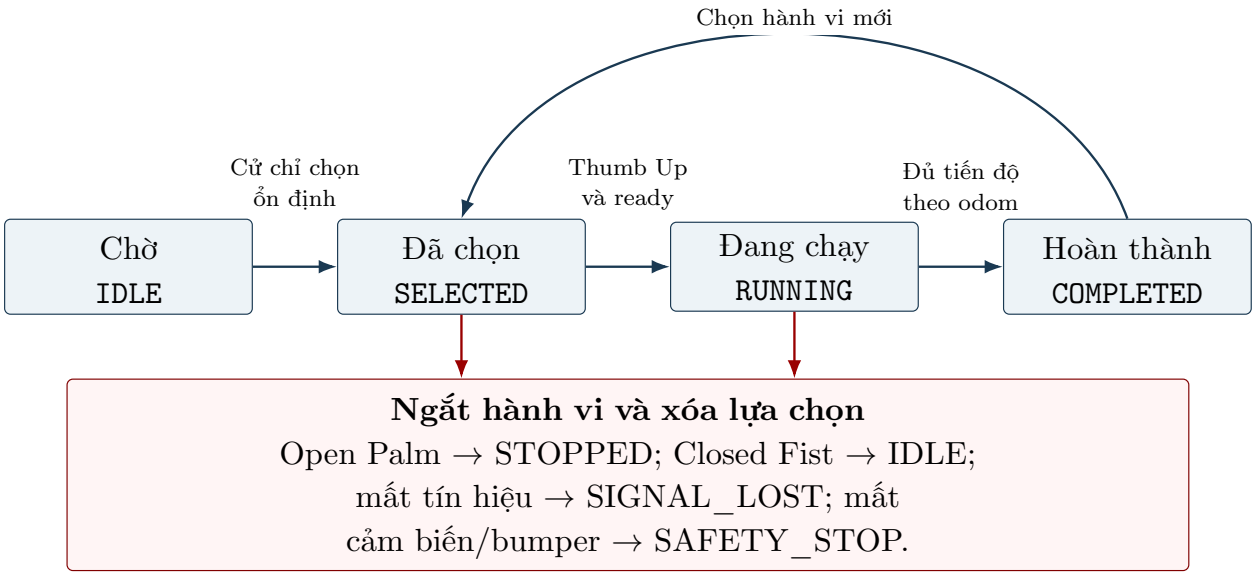
4.1. Ánh xạ cử chỉ và hành vi robot

Cử chỉ	Chuỗi ROS	Hành vi	Điều kiện
Vẫy tay	`Wave`	Chọn quay chào trái–phải	Chờ Thumb Up.
Giơ một ngón	`Pointing_Up`	Chọn bốn chu kỳ tiến–lùi	Chờ Thumb Up.
Giơ hai ngón	`Victory`	Chọn đi một vòng tròn	Chờ Thumb Up.
Giơ ngón cái	`Thumb_Up`	Xác nhận lựa chọn	Đã chọn và robot sẵn sàng.
Bàn tay mở	`Open_Palm`	Dừng, xóa lựa chọn	Không chờ xác nhận.
Nắm tay	`Closed_Fist`	Hủy, trở về chờ	Không chờ xác nhận.

Bảng 3: Bảng ánh xạ cử chỉ tay và hành vi TurtleBot 4.

Ngay cả động tác chào cũng cần được xác nhận. Khi thao tác Wave, người dùng tiếp tục vẫy đến khi thấy `SELECTED: greeting`, rồi chuyển sang Thumb Up; dừng tay ở tư thế bàn tay mở có thể tạo lệnh dừng và xóa lựa chọn vừa có.

4.2. Máy trạng thái và cơ chế xác nhận



Hình 2: Sơ đồ rút gọn chọn–xác nhận–thực hiện và các ngắt ưu tiên. Các điều kiện cảm biến/bumper được kiểm tra liên tục khi đang chạy.

Cử chỉ chọn và xác nhận phải ổn định ít nhất 0,35 giây. Bộ lọc nhớ cử chỉ đã phát sự kiện, nên một cử chỉ giữ liên tục không tạo lệnh mới ở mọi frame. Trong `RUNNING`, một cử chỉ chọn khác không thay thế hành vi hiện tại. Lựa chọn hết hạn sau 10 giây nếu chưa được xác nhận.

Khi xác nhận mà thiếu dữ liệu, trạng thái là `NOT_READY`. Khi hoàn thành, lựa chọn bị xóa và trạng thái là `COMPLETED`. Giữ Thumb Up tiếp tục không đủ để bắt đầu một hành vi mới. `Open_Palm` và `Closed_Fist` được ưu tiên xử lý ngay khi callback nhận cử chỉ hợp lệ, không qua thời gian chờ ổn định của thao tác chọn.

4.3. Thực hiện chuyển động với phản hồi odometry

Vị trí và quaternion từ `/odom` được chuyển thành góc yaw. Mỗi cập nhật bổ sung độ dịch chuyển $\Delta s = \sqrt{\Delta x^2 + \Delta y^2}$ và độ biến thiên góc đã chuẩn hóa. Chuyển pha xảy ra khi tiến độ đạt mục tiêu, không phải do một lệnh sleep cố định.

Tiến–lùi: chuỗi có tám pha, mỗi pha đặt mục tiêu đường đi 0,35 m; vận tốc lệnh đổi dấu luân phiên +0,24 và −0,24 m/s. Một chu kỳ gồm một lượt tiến và một lượt lùi.

Vòng tròn: với bán kính $R = 0{,}60$ m, vận tốc thẳng $v = 0{,}24$ m/s, vận tốc góc được chọn theo

$$\omega = \frac{v}{R} = 0{,}40 \text{ rad/s}, \qquad \sum |\Delta\theta| \geq 2\pi. \tag{3}$$

Robot kết thúc khi tổng biến thiên góc đạt một vòng. Quan hệ $R = v/\omega$ mô tả vận tốc lệnh; kết quả thực tế cần xem trên dữ liệu odom.

Chào: ba pha quay theo thứ tự +0,45 rad, −0,90 rad và +0,45 rad, tốc độ góc lệnh có độ lớn 0,90 rad/s. Các pha tạo chuyển động trái–phải rồi quay về gần hướng ban đầu. Cơ cấu mô phỏng còn chịu giới hạn động học, nên đạt ngưỡng tiến độ không có nghĩa vị trí và góc thực tế không còn vượt thêm trong quá trình giảm tốc.

4.4. Watchdog và điều kiện sẵn sàng

Bộ điều khiển chạy timer 20 Hz theo đồng hồ steady. Tín hiệu cử chỉ hợp lệ cập nhật thời điểm điều khiển gần nhất; vận hành demo bằng cách giữ Thumb Up trong khung hình. Nếu quá 0,65 giây không có cử chỉ hợp lệ, hành vi bị xóa và robot chuyển sang `SIGNAL_LOST`. Bản triển khai không yêu cầu duy nhất Thumb Up để duy trì heartbeat; các cử chỉ hợp lệ khác cũng cập nhật thời điểm nhận, nhưng Open Palm và Closed Fist luôn có tác dụng dừng/hủy riêng.

Điều kiện `ready=True` gồm odom và scan được nhận trong 0,8 giây gần nhất, scan có dữ liệu hợp lệ ở vùng trước và sau, robot không docked, không có bumper và đồng hồ mô phỏng đã bắt đầu. Timeout sử dụng thời gian thực, trong khi timestamp vận tốc sử dụng ROS simulation time. Khi mô phỏng pause làm sensor ngừng cập nhật, tuổi dữ liệu vẫn tăng theo thời gian thực để dẫn đến dừng.

4.5. Kiểm tra vật cản và giới hạn của cơ chế dừng

Khi tiến, node xét vùng LiDAR phía trước; khi lùi, xét vùng phía sau; khi quay tại chỗ, xét toàn bộ scan. Khoảng đo nhỏ hơn 0,35 m theo hướng chuyển động làm robot dừng. Giá trị này là khoảng đo từ LiDAR, không trực tiếp là khoảng cách từ vỏ ngoài robot đến vật cản. Bumper và bước nhảy odometry bất thường cũng gây ngắt.

Cơ chế trên là kiểm tra điều kiện dừng, không thực hiện lập kế hoạch tránh vật cản. Lượt demo được bố trí trong sân trống, dock đặt ở góc $(6, 6)$ và robot xuất phát ở giữa sân. Dock thấp có thể nằm dưới mặt phẳng quét, nên việc bố trí không gian vẫn cần thiết. Video không tiến hành một bài thử va chạm có chủ đích để đánh giá đầy đủ cơ chế này.

Chương 5. Triển khai mã nguồn và vận hành

5.1. Môi trường thực hiện

Thành phần	Cấu hình dùng trong bài
Hệ điều hành và ROS	Ubuntu 24.04, ROS 2 Jazzy, Python 3.12.
Mô phỏng	Gazebo Harmonic, TurtleBot 4 Standard; sân 16×16 m.
Nhận diện	MediaPipe 0.10.21; model `gesture_recognizer.task`; OpenCV 4.6.0; NumPy 1.26.4.
Môi trường Python	`.venv-gesture`, tách khỏi môi trường YOLO của bài trước.
Giao tiếp	ROS domain 76; Gazebo partition tương ứng.
Lượt quay	Ngày 24/10/2026; webcam laptop; một người thực hiện.
Video	MP4 H.264, 1920×1080, 20 fps, 180,05 giây; không âm thanh.

Bảng 4: Môi trường triển khai và thông số bản video nộp.

Tần số 20 fps trong bảng là thông số của video màn hình, không phải tốc độ mô hình MediaPipe. Trong lượt quay, GUI dùng Ogre1, còn server và LiDAR dùng Ogre2. Cách tách renderer giúp thử phương án GUI khác mà không đổi đường render sensor.

5.2. Cấu trúc mã nguồn và chức năng từng file

Listing 2: Cấu trúc package ứng dụng, lược bỏ các file build và cache.

```
turtlebot4_gesture_control/
  turtlebot4_gesture_control/
    gesture_recognition_node.py
    gesture_logic.py
    robot_behavior_node.py
  config/behavior.yaml
  launch/arena.launch.py
  launch/behavior.launch.py
  launch/gui.launch.py
  worlds/gesture_arena.sdf
  scripts/setup_gesture.sh
  test/test_gesture_logic.py
  test/test_arena_world.py
  package.xml
  setup.py
```

5.3. Build và khởi chạy từng terminal

Trong **mỗi terminal**, thiết lập cùng một môi trường trước khi chạy node:

```
cd ~/tb4_ws
export TB4_LAB_DOMAIN_ID=76
source src/turtlebot4_slam_nav/scripts/env.sh
export GZ_IP=127.0.0.1
```

Build và chuẩn bị model khi cài lần đầu:

```
colcon build --symlink-install \
  --packages-up-to turtlebot4_gesture_control
```

File / nhóm file	Vai trò cụ thể
`gesture_recognition_node.py`	Camera, suy luận, vẽ ảnh, heartbeat, publish kết quả.
`gesture_logic.py`	WaveDetector và BehaviorEngine; tách logic để kiểm thử độc lập.
`robot_behavior_node.py`	Liên kết logic với ROS, đọc cảm biến và gửi TwistStamped.
`config/behavior.yaml`	Tốc độ, bán kính, quãng đường, ngưỡng và timeout.
`launch/arena.launch.py`	Tái sử dụng launch robot chính thức; đồng bộ world bridge và đặt dock xa vùng demo.
`launch/behavior.launch.py`	Nạp cấu hình và bật node hành vi với simulation time.
`launch/gui.launch.py`	Chỉ mở GUI để nối lại server đang chạy.
`worlds/gesture_arena.sdf`	Sàn, tường bao và các plugin mô phỏng nền tảng.
`scripts/setup_gesture.sh`	Chuẩn bị môi trường Python riêng và tải model.
`test/*.py`	Kiểm tra logic xác nhận, dừng, tiến độ, Wave và cấu hình world/dock.
`setup.py`, `package.xml`	Khai báo package, phụ thuộc, tài nguyên và executable.

Bảng 5: Giải thích cấu trúc mã nguồn gắn với nhiệm vụ bài thực hành.

```
source install/local_setup.bash
bash src/turtlebot4_gesture_control/scripts/setup_gesture.sh
```

Terminal A khởi chạy Gazebo, terminal B bật hành vi, terminal C bật nhận diện:

Listing 3: Ba lệnh dưới chạy trong ba terminal riêng đã nạp môi trường.

```
# Terminal A
ros2 launch turtlebot4_gesture_control arena.launch.py \
  gui_render_engine:=ogre

# Terminal B
ros2 launch turtlebot4_gesture_control behavior.launch.py

# Terminal C
./.venv-gesture/bin/python -m \
  turtlebot4_gesture_control.gesture_recognition_node \
  --ros-args -p camera_index:=0 -p confidence:=0.65 \
  -p max_fps:=15.0
```

Trong sân mặc định của bài, robot không bắt đầu trên dock nên không cần Undock. Người dùng chờ `ready=True` trước khi chọn và xác nhận hành vi. Sau khi đổi YAML tốc độ, cần khởi chạy lại node hành vi để nạp thông số; không suy ra cấu hình đang chạy chỉ từ file đã sửa trên đĩa.

5.4. Kiểm tra các topic và thông số

```
ros2 topic type /cmd_vel
ros2 topic info /cmd_vel -v
ros2 topic hz /scan
ros2 topic hz /odom
ros2 topic echo /detected_gesture
ros2 topic echo /robot_behavior_state \
```

```
  --qos-durability transient_local
```

Các lệnh theo dõi chạy riêng, dừng bằng Ctrl+C trước khi nhập lệnh tiếp theo. Chúng không được dùng để publish vận tốc trong video demo. Nếu GUI đóng nhưng server vẫn hoạt động, mở lại cửa sổ bằng `ros2 launch turtlebot4_gesture_control gui.launch.py` trong terminal cùng domain. Nếu server đã dừng thì cần chạy lại arena.

Thông số	Giá trị	Ý nghĩa
`stable_time`	0,35 s	Ổn định cử chỉ chọn/xác nhận.
`signal_timeout`	0,65 s	Mất cử chỉ hợp lệ dẫn đến dừng.
`sensor_timeout`	0,80 s	Giới hạn tuổi odom/scan.
`selection_timeout`	10,0 s	Thời hạn lựa chọn chờ xác nhận.
`motion_timeout`	180,0 s	Giới hạn thời gian thực cho một hành vi.
`linear_speed`	0,24 m/s	Tốc độ thẳng lệnh.
`angular_speed`	0,90 rad/s	Tốc độ góc lệnh khi chào.
`stroke_distance`	0,35 m	Tiến độ mục tiêu mỗi lượt tiến/lùi.
`circle_radius`	0,60 m	Bán kính dùng tính vận tốc góc.
`greeting_angle`	0,45 rad	Biên góc cơ sở của chuỗi chào.
`obstacle_distance`	0,35 m	Ngưỡng khoảng đo LiDAR.

Bảng 6: Cấu hình hành vi được dùng trong lượt quay ngày 24/10/2026.

Chương 6. Kết quả thực nghiệm và đánh giá

6.1. Kịch bản, cách thu dữ liệu và giới hạn đánh giá

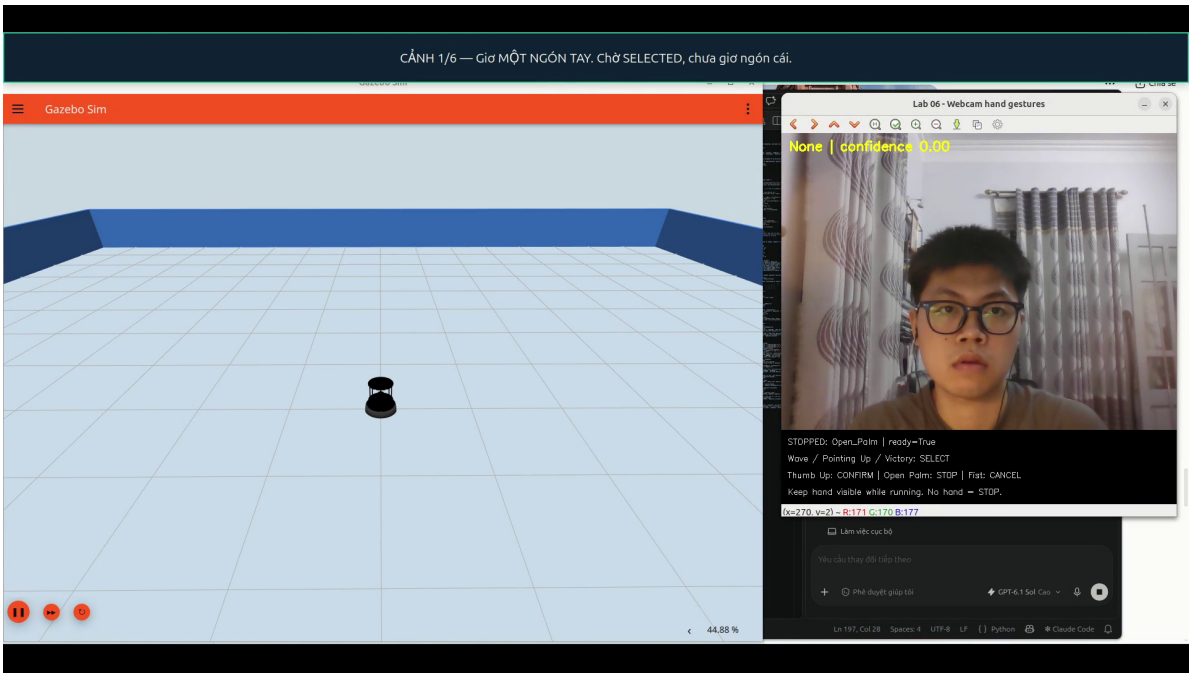
Bộ dữ liệu thực nghiệm [9] gồm video ghi đồng thời Gazebo, webcam và thanh nhắc các bước. Người dùng trực tiếp thực hiện cử chỉ; công cụ quan sát chỉ subscribe, ghi sự kiện và odometry. Thứ tự thực hiện là tiến–lùi, vòng tròn, chào, dừng bằng Open Palm, hủy bằng Closed Fist và mất tín hiệu khi tay ra khỏi ảnh. Việc sắp cửa sổ hoặc chọn vùng ghi màn hình thuộc bước chuẩn bị, không dùng để điều khiển robot.

Mẫu thực nghiệm gồm một video 180,05 giây với một người dùng. Các mốc sự kiện dùng đồng hồ monotonic của công cụ quan sát; hình được lấy từ video hoặc ảnh có chú thích do node nhận diện publish. Sai khác thời điểm giữa ảnh, callback ROS và chuyển trạng thái có thể xuất hiện do chu kỳ cập nhật và bộ đệm. Vì vậy, một ảnh nhận cử chỉ chưa chắc đã hiển thị ngay trạng thái hành vi mới.

Dữ liệu giữ nguyên hai đoạn phải làm lại. Báo cáo không suy ra tỷ lệ chính xác 100% từ việc sáu cảnh cuối cùng đạt. Không có bộ nhãn ground truth theo từng frame, nên cũng không lập ma trận nhầm lẫn hoặc đếm tất cả lần nhận đúng/sai trong video. Số 884 trong dữ liệu ghi là số ảnh preview được lấy mẫu, không phải tổng số frame của video hoặc số lượt suy luận AI.

6.2. Kết quả nhận diện sáu cử chỉ

Cả sáu nhãn yêu cầu đã xuất hiện trong lượt quay. Hình 4 là các ảnh thật được node lưu khi lần đầu nhận được từng nhãn; phần hướng dẫn dưới ảnh được lược bỏ khi trình bày để tập



Hình 3: Màn hình thực nghiệm gồm robot trong Gazebo và webcam có nhãn, confidence, trạng thái. Thanh phía trên hướng dẫn người dùng thực hiện từng cảnh.

trung vào bàn tay và kết quả phân loại. Bảng 7 liệt kê confidence của các mẫu đó, không phải confidence trung bình hay độ chính xác toàn lượt.



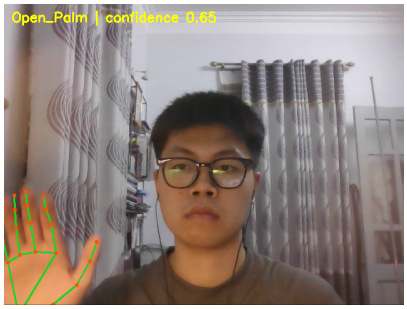
(a) Pointing Up



(b) Victory



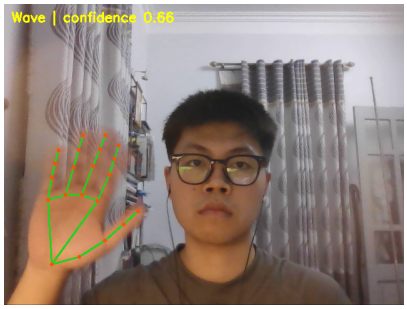
(c) Thumb Up



(d) Open Palm



(e) Closed Fist



(f) Wave

Hình 4: Sáu cử chỉ nhận từ webcam của người dùng trong chính lượt demo. Wave sử dụng thêm lịch sử chuyển động, nên một ảnh riêng chỉ minh họa nhãn tại thời điểm đó.

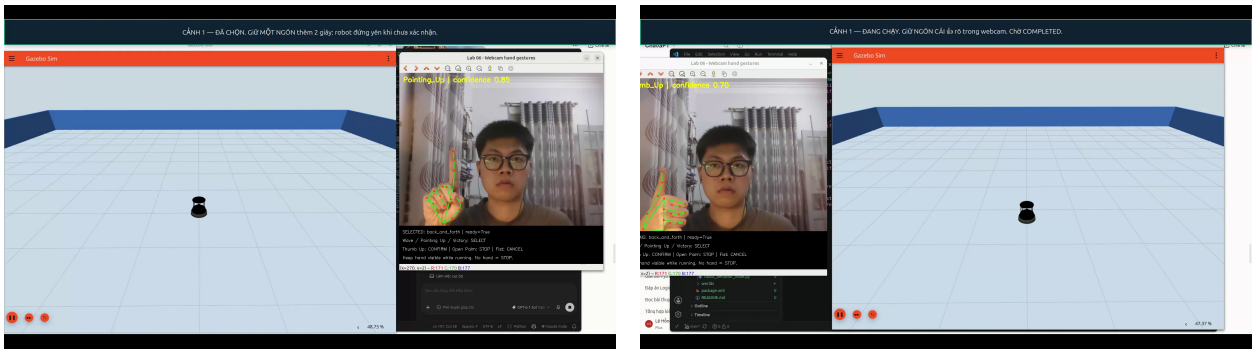
Các mẫu Thumb Up và Open Palm gần ngưỡng cho thấy khoảng dự phòng nhỏ: một biến đổi nhẹ về tư thế hoặc hình ảnh có thể đưa đầu ra về `None`. Ngược lại, một mẫu Closed Fist có điểm cao không chứng minh nắm tay luôn dễ nhận hơn các cử chỉ khác; cần nhiều lượt thử độc lập để rút ra so sánh đó.

Nhãn	Confidence mẫu	Diễn giải
`Pointing_Up`	0,7967	Mẫu một ngón tay được chấp nhận.
`Victory`	0,8582	Mẫu hai ngón chữ V.
`Thumb_Up`	0,6525	Mẫu đầu tiên chỉ cao hơn ngưỡng 0,65 một ít.
`Open_Palm`	0,6534	Mẫu bàn tay mở, có phần tay gần biên ảnh.
`Closed_Fist`	0,9742	Mẫu nắm tay tại thời điểm nhận diện.
`Wave`	0,6595	Điểm Open Palm cộng với điều kiện chuyển động đã đạt.

Bảng 7: Confidence của một ảnh mẫu cho mỗi nhãn, lấy từ log và ảnh của lượt quay.

6.3. Chọn hành vi và xác nhận trước khi chuyển động

Trong cảnh đầu tiên, người dùng giữ Pointing Up khi trạng thái đã là `SELECTED`; robot vẫn đứng yên. Sau đó Thumb Up được nhận ổn định và trạng thái chuyển sang `RUNNING`. Hình 5 cho thấy hai thời điểm trong cùng chuỗi thao tác. Đây là minh chứng trực tiếp cho việc tách lựa chọn khỏi thực thi.



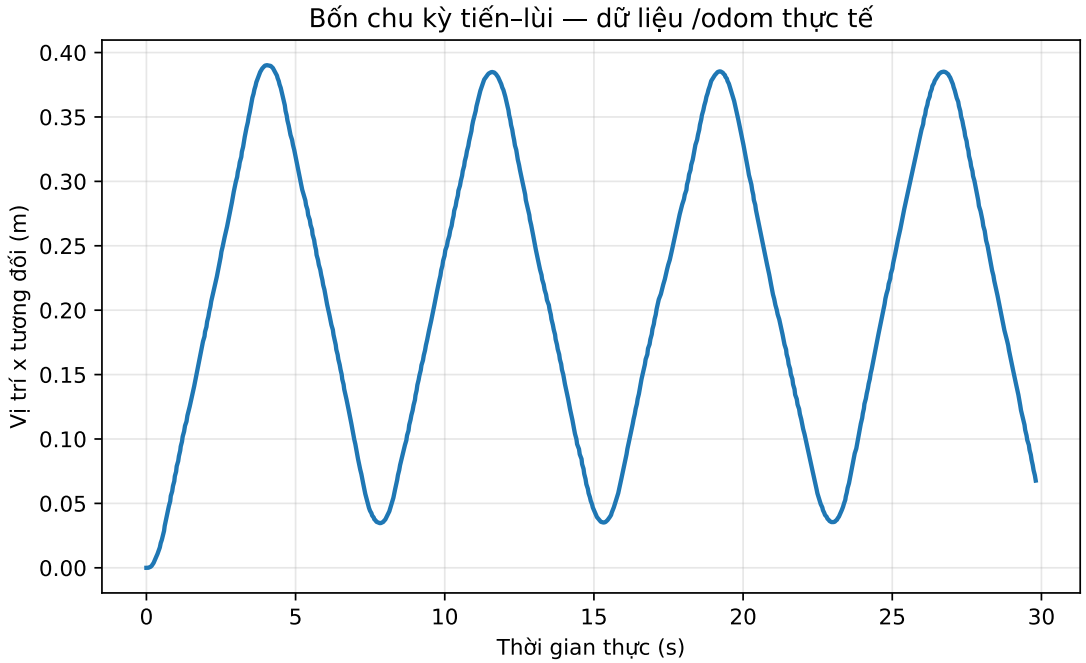
(a) Đã chọn, chưa xác nhận. **(b)** Đã xác nhận, đang chạy.

Hình 5: Minh chứng cơ chế chọn–xác nhận. Chi tiết trạng thái có thể đối chiếu trực tiếp trên video và log sự kiện.

6.4. Bốn chu kỳ tiến–lùi

Log ghi nhận đủ tám pha `RUNNING` theo thứ tự và trạng thái `COMPLETED` ở khoảng giây 47,22 của lượt ghi. Khoảng từ bắt đầu chạy đến hoàn thành là 29,85 giây theo đồng hồ thực. Đồ thị Hình 6 thể hiện bốn đỉnh tương ứng bốn lượt tiến, xen kẽ bốn lượt lùi; không suy ra số chu kỳ chỉ từ một ảnh robot đứng ở cuối đường.

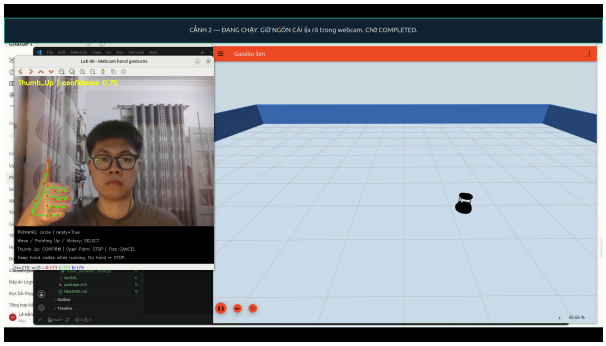
Biên tiến đầu tiên trên đồ thị khoảng 0,390 m, lớn hơn mục tiêu pha 0,35 m. Tiến độ trong triển khai được cộng từ độ dịch chuyển dương, còn vận tốc thay đổi theo các giới hạn của controller nền tảng; chương trình chưa điều khiển vị trí bằng sai số có dấu và chưa giảm tốc theo quãng đường còn lại. Vì vậy, chuyển pha tại ngưỡng không đồng nghĩa robot đổi chiều tức thời hoặc trở về đúng tọa độ ban đầu. Mẫu cuối của khoảng RUNNING–COMPLETED còn lệch khoảng 0,068 m so với mẫu đầu; đây không phải phép đo sai số vị trí sau khi robot đã dừng ổn định hoàn toàn. Kết quả đạt yêu cầu số chu kỳ, nhưng còn hạn chế về độ chính xác điểm đảo chiều và điểm trở về.



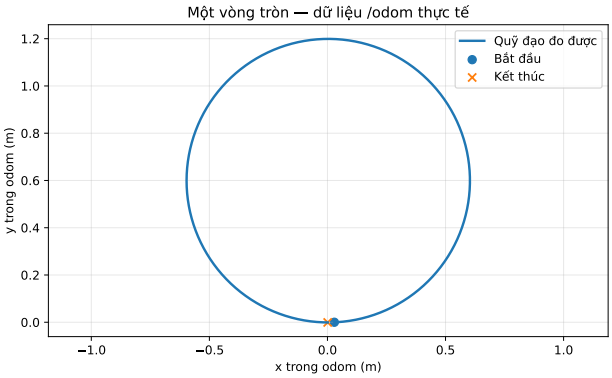
Hình 6: Vị trí x tương đối của robot trong khoảng chạy tiến–lùi, lấy từ dữ liệu `/odom`.

6.5. Quỹ đạo vòng tròn

Cảnh vòng tròn hoàn thành sau 35,70 giây thực và kết thúc ở khoảng giây 92,87 của lượt ghi. Hình 7 dựng từ các cặp (x, y) thu được; không thay dữ liệu đo bằng đường tròn lý tưởng. Khớp đường tròn theo bình phương tối thiểu trên chính odometry cho bán kính xấp xỉ 0,600 m, phù hợp bán kính lệnh.



(a) Chuyển động quan sát trong Gazebo.



(b) Quỹ đạo đo trong hệ odom.

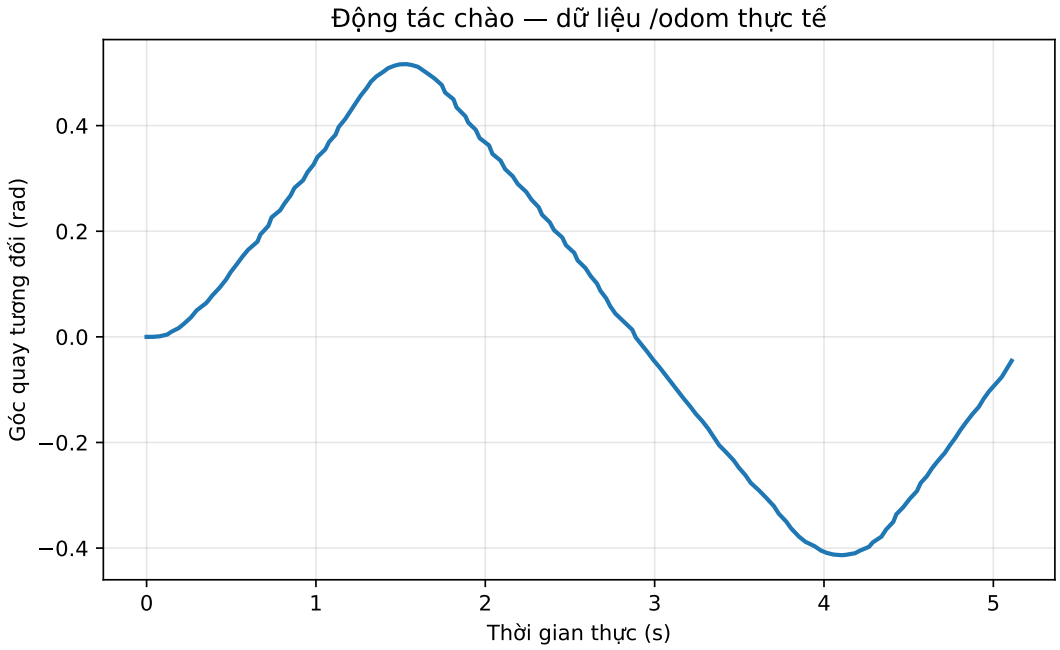
Hình 7: Thực hiện một vòng tròn sau khi chọn Victory và xác nhận.

Khoảng cách giữa mẫu đầu và cuối trong khoảng lấy dữ liệu là khoảng 0,028 m; biến thiên yaw có dấu sau unwrap khoảng 6,279 rad, gần 2π. Các giá trị này đánh giá tính khép kín trong hệ odom, chưa phải sai số đối với ground truth của Gazebo vì lượt quay không lưu riêng ground truth để so sánh.

Thời gian $2\pi/0{,}40 \approx 15{,}71$ giây là giá trị lý tưởng theo vận tốc góc lệnh trong thời gian mô phỏng. Không đối chiếu trực tiếp nó với 35,70 giây đồng hồ thực để kết luận robot sai tốc độ: còn có quá trình tăng tốc, cập nhật điều khiển và tốc độ chạy vật lý so với thời gian thực. Lượt này chưa đo đầy đủ real-time factor theo thời gian để tách riêng từng ảnh hưởng.

6.6. Động tác chào từ Wave

Người dùng vẫy bàn tay mở, hệ thống nhận Wave và chọn `greeting`. Sau Thumb Up, robot thực hiện đủ ba pha quay và báo `COMPLETED`, với thời gian chạy khoảng 5,15 giây thực. Đồ thị góc tương đối thể hiện chuyển động sang trái, sang phải rồi trở về gần hướng đầu. Đỉnh



Hình 8: Góc yaw tương đối theo odometry trong hành vi chào.

dương khoảng 0,516 rad, cực tiểu khoảng −0,414 rad và mẫu cuối còn khoảng −0,045 rad so với hướng đầu. Các giá trị cho thấy động tác chào đúng thứ tự nhưng chưa đạt điều khiển góc chính xác tuyệt đối. Có một lần lựa chọn Wave bị xóa khi nhãn trở về Open Palm trước khi xác nhận thành công; đoạn này được giữ trong video, không loại bỏ khỏi phần nhận xét.

6.7. Dừng, hủy và mất tín hiệu

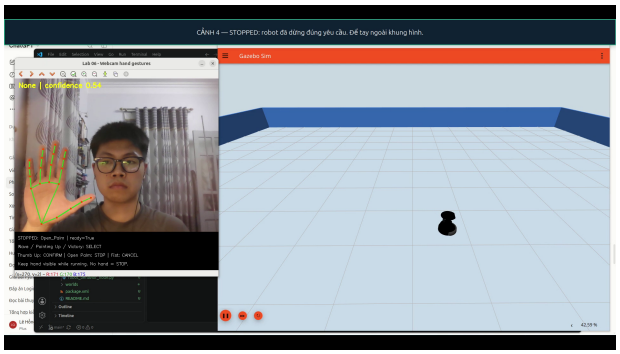
Trong cảnh Open Palm, robot đang chạy vòng tròn rồi chuyển sang `STOPPED`. Trong cảnh Closed Fist, robot dừng và về `IDLE`. Khi người dùng đưa tay ra ngoài webcam, trạng thái trở thành `SIGNAL_LOST`. Hình 9 minh họa các trạng thái này; việc dừng được đối chiếu với video và log, không chỉ với thanh nhắc động tác.

Timeout 0,65 giây là khoảng tính từ cử chỉ hợp lệ gần nhất tại node hành vi, không phải phép đo trực tiếp từ lúc người dùng bắt đầu đưa tay ra. Trễ webcam, tuổi ảnh, timer và phản ứng của controller ảnh hưởng thời gian quan sát. Video chưa có phép đo đồng bộ để công bố độ trễ dừng đầu–cuối chính xác.

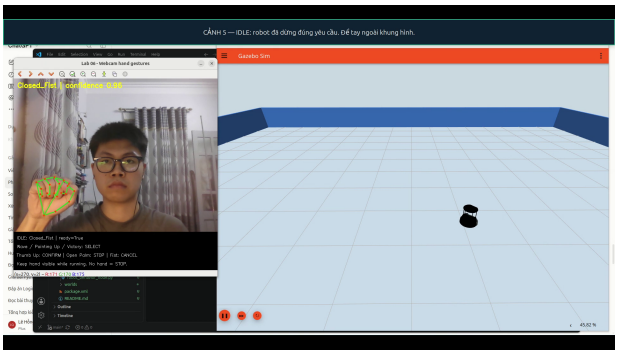
Sau dừng/hủy, mã nguồn xóa lựa chọn; logic này được kiểm thử tự động. Tuy nhiên, lượt video được giữ không có cảnh riêng giơ Thumb Up sau STOPPED hoặc SIGNAL_LOST để minh chứng không tự chạy lại. Vì vậy, báo cáo phân biệt tính năng thể hiện trong mã/kiểm thử với thao tác đã được quay thực tế.

6.8. Bảng tổng hợp kết quả và các lần gián đoạn

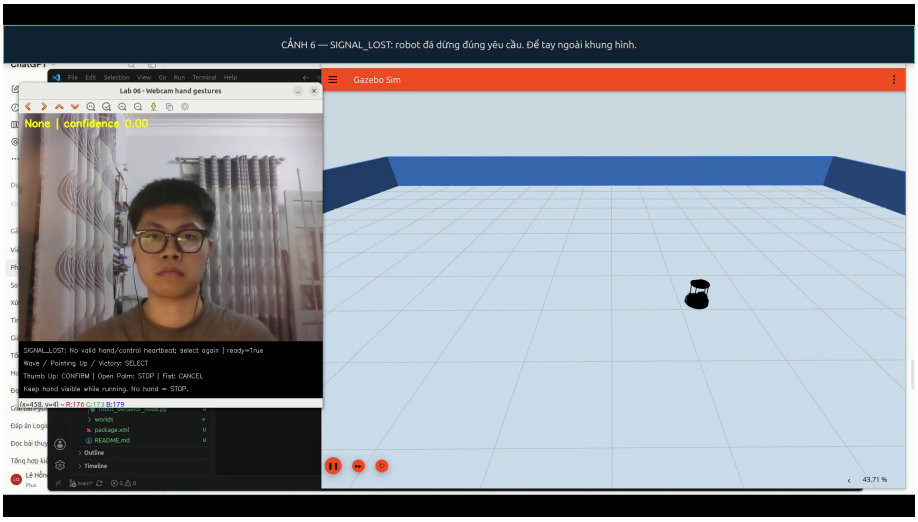
Sáu cảnh đều đạt trạng thái đích sau quá trình thao tác. Hai đoạn gián đoạn gồm: ở cảnh 3, bàn tay mở đứng yên làm lựa chọn chào bị xóa; ở cảnh 4, quá trình chuyển từ Thumb Up sang bàn tay mở có khoảng không nhận diện đủ dài, robot dừng bằng SIGNAL_LOST trước



(a) STOPPED sau Open Palm.



(b) IDLE sau Closed Fist.



Hình 9: Minh chứng dừng/hủy và dừng do mất bàn tay. Trong ảnh STOPPED, nhãn ở frame hiện tại đã dao động về None; trạng thái lưu lại lý do dừng trước đó là Open Palm.

Cảnh	Nội dung	Trạng thái đạt	Mốc đạt (s)	Gián đoạn trước đó
1	Một ngón, xác nhận, bốn chu kỳ tiến–lùi	`COMPLETED`	47,22	0
2	Victory, xác nhận, một vòng tròn	`COMPLETED`	92,87	0
3	Wave, xác nhận, động tác chào	`COMPLETED`	115,22	1
4	Open Palm khi đang chạy	`STOPPED`	144,57	1
5	Closed Fist khi đang chạy	`IDLE`	157,68	0
6	Đưa tay ra khỏi webcam	`SIGNAL_LOST`	171,52	0

Bảng 8: Kết quả sáu cảnh. Mốc đạt là thời điểm xấp xỉ trong lượt ghi; số gián đoạn là số đoạn bị ngắt và phải làm lại, không phải thống kê lỗi trên từng frame.

khi nhận được Open Palm. Lần thứ hai, người dùng chuyển tư thế dứt khoát hơn và đạt STOPPED đúng kịch bản. Không quy hai đoạn này thành một con số accuracy vì đơn vị thử và nhãn ground truth chưa được định nghĩa như một thí nghiệm phân loại.

6.9. Phân tích vấn đề đã gặp và cách xử lý

Wave thường trở thành Open Palm. Cấu hình ban đầu yêu cầu ba lần đảo chiều trong 1,8 giây và theo dõi phần gần cổ tay. Khi người dùng chỉ vẫy quanh cổ tay, đặc trưng dịch chuyển nhỏ và dễ không đạt. Bản dùng trong video theo dõi đầu ngón tay, cửa sổ 3 giây và hai lần đảo chiều, đồng thời xóa lịch sử khi tay đã đứng yên. Việc điều chỉnh cải thiện tính

thao tác nhưng vẫn phải duy trì phân biệt bàn tay mở dùng để dừng.

Mất tín hiệu dù tay còn trong ảnh. Đây là khác biệt giữa phát hiện bàn tay và phân loại cử chỉ hợp lệ. Thay đổi góc tay hoặc tư thế trung gian có thể làm nhãn thành None; nếu kéo dài quá timeout thì robot dừng đúng logic. Không tăng timeout chỉ để che vấn đề mà không đánh giá ảnh hưởng đến thời gian dừng khi thực sự mất điều khiển.

Robot báo NOT_READY. Đã ghi nhận một phiên Gazebo ở domain 74 trong khi node hành vi và webcam ở domain 76. Kiểm tra trực tiếp cho thấy domain 74 có scan/odom/clock, còn domain 76 không có. Đồng bộ ba terminal về domain 76 giải quyết nguyên nhân đó. Không nên hạ điều kiện cảm biến để buộc robot chạy khi thiếu dữ liệu.

GUI Gazebo tự đóng. Log có GUI thoát mã 139 và báo lỗi material/shader của Ogre2; stack trace Qt được cung cấp chưa đủ để kết luận chính xác lỗi C++. Phương án dùng GUI Ogre1, giữ sensor/server Ogre2 được thử riêng; GUI còn chạy sau khoảng 359 giây theo dõi, scan/odom vẫn có dữ liệu. Chức năng mở lại GUI được bổ sung và xác nhận đồng hồ mô phỏng không reset khi kết nối lại. Việc thay renderer phù hợp với hướng xử lý render nêu trong tài liệu Gazebo [7]; đây chưa phải chứng minh loại trừ mọi crash khi tương tác lâu dài.

6.10. Đánh giá kiểm thử và phạm vi kết luận

Bộ kiểm thử tự động đã ghi nhận 18 test đạt, bao gồm chọn–xác nhận, chống lặp, dừng/hủy, mất heartbeat, cảm biến/vật cản, đủ pha chuyển động và các điều kiện Wave. Các test cấu hình còn kiểm tra bridge đúng world và vị trí dock khớp TF. Các test này cung cấp bằng chứng logic, không thay thế lượt demo bằng tay thật.

Lượt quay cho thấy chức năng vận hành với người dùng thật, nhưng chưa đủ đánh giá tổng quát theo ánh sáng, tốc độ tay, khoảng cách và nhiều người. Chưa có dữ liệu toàn bộ suy luận theo frame để tính FPS thực tế hoặc ma trận nhầm lẫn. Cũng chưa có bài thử hệ thống với vật cản động, LiDAR bị che hay các tình huống tiếp xúc có chủ đích. Kết luận của báo cáo được giới hạn ở các chức năng và dữ liệu đã mô tả.

Chương 7. Kết luận và hướng phát triển

7.1. Kết quả đạt được

Bài thực hành đã triển khai chuỗi webcam–nhận diện–ROS 2–TurtleBot 4 với hai node ứng dụng tách biệt. Hệ thống sử dụng năm cử chỉ tĩnh có sẵn và một cử chỉ Wave dựa trên lịch sử chuyển động; gửi kết quả qua topic, hiển thị nhãn/confidence và áp dụng cơ chế chọn–xác nhận trước chuyển động.

Trong video ngày 24/10/2026, robot hoàn thành bốn chu kỳ tiến–lùi, một vòng tròn và động tác chào, đồng thời thể hiện dừng bằng Open Palm, hủy bằng Closed Fist và dừng khi mất tín hiệu. Ảnh cùng dữ liệu odometry cung cấp minh chứng độc lập với lời mô tả. Hai lần phải thao tác lại được giữ để phản ánh hạn chế của nhận diện và giao thức điều khiển.

7.2. Hạn chế và hướng cải tiến

Cải tiến ưu tiên là xây dựng bộ thử có số lần lặp xác định, nhiều người và nhiều điều kiện ảnh; gán nhãn độc lập để tính tỷ lệ đúng, bỏ sót và nhầm lớp. Với Wave, có thể chuẩn hóa biên độ theo kích thước bàn tay hoặc đánh giá bộ phân loại chuỗi động tác thay cho một ngưỡng theo chiều rộng ảnh cố định.

Về điều khiển, cần thử kết thúc pha theo độ dịch chuyển có dấu, giảm tốc theo sai số còn lại và đo tư thế sau khi ổn định để giảm vượt ngưỡng. Cần ghi đồng bộ thời điểm cử chỉ, phát lệnh và vận tốc thực để đo độ trễ dừng; đo FPS suy luận trên các frame mới thay vì lấy tần số publish. Cuối cùng, nên bổ sung video riêng cho kiểm tra không tự khởi động lại sau dừng/mất tín hiệu và thử các điều kiện cảm biến trong một quy trình thực nghiệm có kiểm soát.

Tài liệu tham khảo

[1] Google AI Edge, *Gesture recognition task guide*. https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer. Truy cập 24/10/2026.

[2] Google AI Edge, *Gesture recognition guide for Python*. https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer/python. Truy cập 24/10/2026.

[3] OpenCV, *Getting Started with Videos*. https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html. Truy cập 24/10/2026.

[4] Open Robotics, *Understanding topics*, ROS 2 Jazzy documentation (bản nguồn). https://github.com/ros2/ros2_documentation/blob/jazzy/source/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.rst. Truy cập 24/10/2026.

[5] ROS 2 Common Interfaces, *TwistStamped message*, nhánh Jazzy. https://github.com/ros2/common_interfaces/blob/jazzy/geometry_msgs/msg/TwistStamped.msg. Truy cập 24/10/2026.

[6] TurtleBot 4 User Manual, *TurtleBot 4 Simulator*. https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_simulator.html. Truy cập 24/10/2026.

[7] Gazebo, *Troubleshooting – Harmonic*. https://gazebosim.org/docs/harmonic/troubleshooting/. Truy cập 24/10/2026.

[8] Mã nguồn thực hiện bài thực hành, package `src/turtlebot4_gesture_control`; cấu hình `config/behavior.yaml` và các node được đối chiếu khi viết báo cáo.

[9] Bộ dữ liệu thực nghiệm 24/10/2026: video `demo_turtlebot4_gestures.mp4`, `events.jsonl`, `odometry.jsonl`, `results.csv` và ảnh minh chứng trong thư mục lượt quay.

Phụ lục: Bộ minh chứng thực nghiệm

Bộ minh chứng gồm video MP4 180,05 giây, ảnh webcam và trạng thái, `events.jsonl`, `odometry.jsonl`, `results.csv`. Các file được lưu trong lượt quay ngày 24/10/2026 và gói `lab06_demo_2026-10-24.zip`. Hướng dẫn biên dịch pdfLaTeX và thay thông tin bìa nằm trong README của gói mã nguồn báo cáo.